Problem 1

1. Start

2. Check input of list
   - If empty or length is 0, warn the user and return empty list

3. New empty list created to store new order of original elements
   - perform modulus on k with length of original list
   - add to new list starting from initial - result of modulus

4. Return the new list.

Problem 2

1. Start

2. Determine the total distance from the start position and end position

3. If distance is completely divisible by the amount frog jumps at a time, output the result of that division.
   - distance / amount frog jumps

4. Otherwise, take the integer value of division and add 1

Problem 3:

1. Start

2. Check if input is empty / list is length 0
   - If true then warn user to input list

3. Create counter starting from 0

4. From range 1 to length of list, if that number exists, counter goes up by 1
   - A loop that checks if it exists in list. True means counter goes up. Next number goes through same evaluation

5. If counter = length of list, return 1. Else return 0

Problem 4:

1. Start

2. Create function variable
   - E = [] an empty list for cars travelling east
   - W = [] an empty list for cars travelling west
   - pair counter to record pairs of cars. Set value to 0

3. Check input
   - If empty or length equal 0, return a message requesting valid input
   - If element in list equal 0, add to E[]
   - If element in list equal 1, add to W[]
   - If neither, indicate error of input. Set value to NoneType

4. Compare lists for cars passing pairs
   - If the elements of E list is smaller than elements of W record as pairs

Problem 5:

1. Start

2. Check list input
   - If empty or length equal to 0 warn user to input a list

3. Create a counter variable starting from 1

4. While counter exists in the list, add one to counter
   - Counter will count up until it is no longer in the list
     Since it counts up 1 at a time, the value it stops at will always be the smallest integer not present in the list.